

コンピュータビジョン

担当: 井尻 敬

○ 講義の概要:

画像処理は、産業・自然科学・エンタテインメントなど、多種多様な分野の発展に関わる非常に重要な技術です。2年生時のデジタルメディア処理では、画像処理の基本である『画像データ構造・画像撮影方法・各種フィルタ・拡大縮小・補間』などについて紹介しました。この内容をさらに発展させ、**本コンピュータビジョンでは、計算機が画像を認識する手法**について紹介します。具体的には、画像から目的部分を切り抜く領域分割、画像の特徴を計算機が理解できる形で記述する特徴抽出、および、抽出した特徴を用いて画像を識別するパターン認識について解説します。また、講義後半では、深層学習を用いた画像処理についても紹介します。

本講義にて解説する技術に関して、コーディング可能な深さで理解できるよう、ソースコードを交えながら詳細な技術解説を行ないます。また、Pythonを用いたプログラミング演習を通して画像処理手法のより深い理解を目指します。講義資料は井尻のweb page (takashijiri.com) に事前にアップロードします。

○ 達成目標:

1. 領域分割 – 画像の領域分割法について主要なアルゴリズムを説明・実装できる
2. 特徴抽出 – 画像認識に必要な特徴抽出の基礎を説明・実装できる
3. パタレコ – 画像に対するパターン認識（顔認識など）の基礎やアルゴリズムを説明・実装できる

○ 成績評価:

毎回の小テスト（15%）、筆記試験（45%）、プログラミング課題（40%）に基づき評価します。

○ 講義資料:

講義で紹介した資料・ソースコードは可能な限りWeb上に公開します。

<https://takashijiri.com/classes>

<https://github.com/Takashijiri/PythonOpenCVPractice>

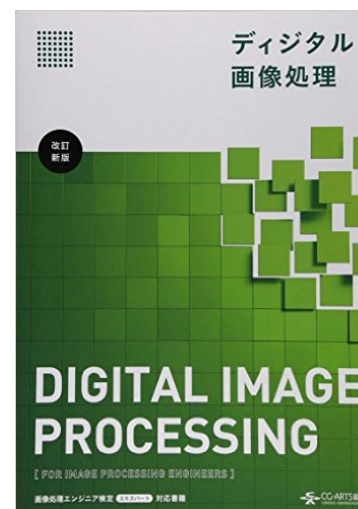
○ 場所:

豊洲キャンパス

- 座学: 未定
- 演習: 未定

○ その他:

- あめ・がむ・飲み物などをご自由に。お手洗い許可不要。スマホ通話は教室の外で。
- 周囲に迷惑となる程度の私語は控えて下さい（小さな声で教え合うのはOK）（本学の私担当の講義で私語が問題になったことは無いですが、念の為）。
- 小テストの解答データの共有やコピー、プログラミング課題の解答データの共有やコピーペーストは、カンニングと同等の扱いとするので絶対に行わないでください。（わからない部分の教え合いは奨励します。）



参考書

- CG-Arts協会（画像情報教育進行委員会）
- デジタル画像処理[改訂新版] 大型本
- 日本語で読める画像処理の教科書です
- 画像や例が多く入門者には最適だと思います
- 網羅性が非常に高い反面、初学者にとっては説明が少ない部分もある → 講義中に丁寧に解説します

※基本的に、スライド資料を中心に講義を進めます

小テスト/筆記試験について

小テスト

- 1~8回目の講義内にて実施
- 実施方法：scombz
- 解答期間：**講義の最後20分~30分** ※ただし「初回」「講義が長引いた場合」「scombzのトラブル」「ややこしい計算問題がある回」は解答期間を延長します
- 主にその日の内容から出題予定（一部次週の内容を含む）
- 持ち込みOK、テキスト参照OK、生成AI不可、相談不可

筆記試験

- 第9回目に対面実施
- 紙ベースで実施予定
- 持ち込み不可
- 小テストの類題が50%以上で構成する予定
- コース外履修者で対面参加できないことが決まっている方は事前に連絡

復習: デジタルメディア処理

本講義はデジタルメディア処理（2年後期）の知識を前提とする

- 画像とは
- 画像の変形とアファイン変換
- フーリエ変換 (FFT)
- フィルタ処理
- 畳み込み
- 逆畳み込み

※未履修の方は適宜独習してください

※過去資料は <http://takashijiri.com/classes/index.html>

講義を理解するために前提とする数学知識

- 高校までの数学
- 線形代数全般：ベクトル、ベクトルの内積、行列、行列の積、固有値固有ベクトル
- 最適化数学（主に最急降下法を利用）
- 畳み込みとフーリエ変換

復習をお願いします。

Contents

- **本講義で取り扱う問題の概要（特徴ベクトルとは）**
- ニューラルネットワーク
- 各回で取り扱う内容の紹介

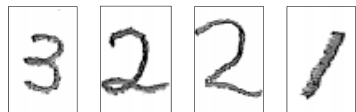
本講義で扱う課題の概要 (特徴ベクトルとは)

この画像には何が写っている？



なぜそうわかったのでしょうか？
おなじような判断をするプログラム
を作成できますか？

画像を数字の「1」「2」「3」「4」に分類せよ



なぜそう分類したのでしょうか？
その分類をするプログラムを設計してみてください

例えば以下のようなのはどう？

- + 線を抽出
 - + 大きな曲がり角で分割
 - + 直線の曲線に分解
 - + 直線1本なら1、曲線と直線なら2、曲線2個なら3、、、、
- ちょっと筋が悪そう。。

※ プログラミング課題で扱います



画像をりんご・バナナ・みかんに分類せよ 1

ここで
+分類先の3つのクラス
+サンプル画像（正解画像）
が与えられている下での分類
を考えてみます

ラベル付き正解画像

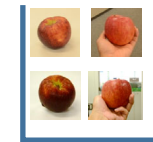


りんご バナナ みかん

分類対象の未知画像



以下のように3種のビンに入れるイメージ



りんご



バナナ



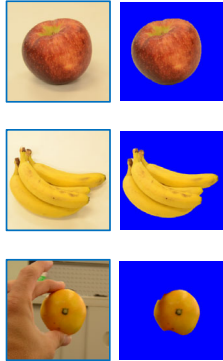
みかん

この問題を解いてみます（詳細は第6回）
アイデア：
色（赤 v.s. 黄色）、形（丸い v.s. 細長い）
で分類できそう！

画像をりんご・バナナ・みかんに分類せよ 2

前処理:

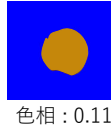
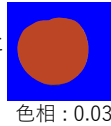
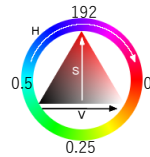
正解画像の前景領域を抽出



平均の色を取得:

前景領域の平均の色相を利用して

色相 → 色味のこと
下図は色相環

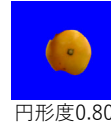
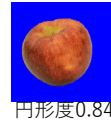


円形度を取得:

領域が円なら1.0になる円形度を計算

$$\frac{A}{L^2/4\pi}$$

A: 領域の面積
L: 領域の周囲長
 $L^2/4\pi$: 周囲長Lの円の面積

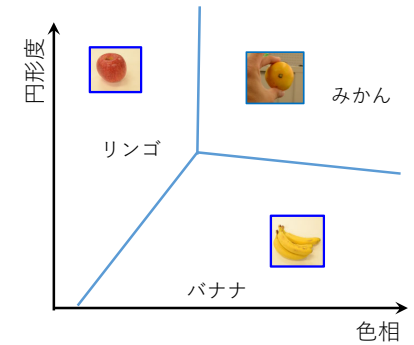


画像をりんご・バナナ・みかんに分類せよ 3

正解画像について(色相, 円形度)を計算すると...

	(色相, 円形度)
りんご	(0.03, 0.84)
バナナ	(0.12, 0.52)
みかん	(0.11, 0.80)

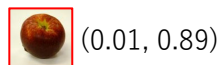
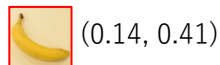
画像を二次元空間に配置できる



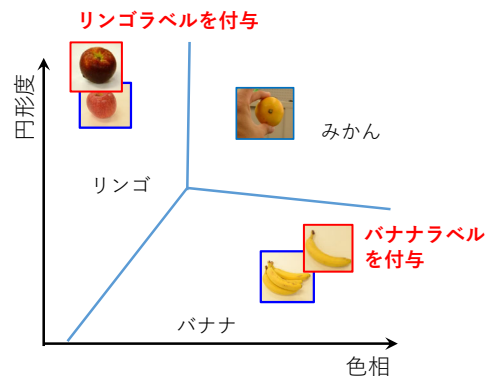
色相-円形度 空間に正解をプロット
正解データの分布に基づいて分割しておく

画像をりんご・バナナ・みかんに分類せよ 4

未知画像についても(色相, 円形度)を計算

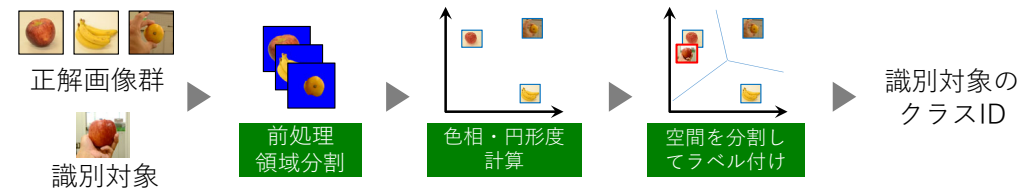


先の空間の分割結果を利用すると分類が可能



※ 数値は適当に作った嘘データです。

先程の手順を振り返る

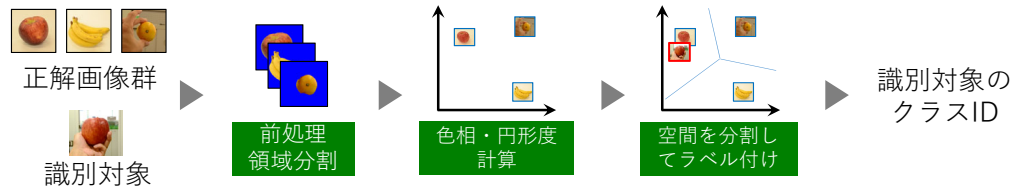


今回の例で計算した(色相, 円形度)を**特徴ベクトル**と呼び、特徴ベクトルが配置される空間を**特徴空間**と呼ぶ ※色相・円形度それぞれは特徴量と呼ばれる

- 特徴ベクトルを導入することで画像同士の比較(距離計算)が可能になる
- 特徴ベクトルは問題に依存して設計する必要がある

古典的なComputer Visionでは「この特徴ベクトルの設計」と「正解データの分布から空間を分割する方法」が重要な研究対象だった。

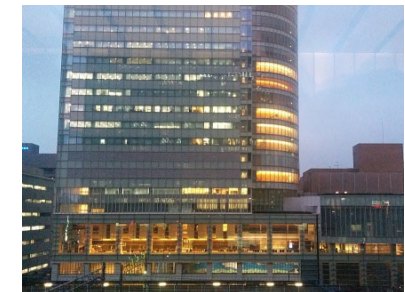
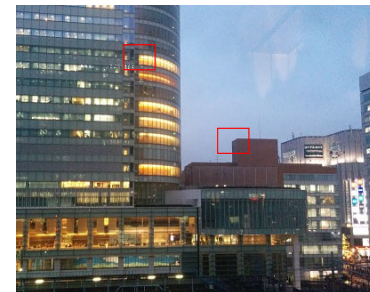
先程の手順を振り返る



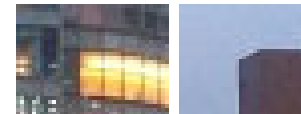
問) 以下について考えてみてください

- 特徴ベクトルはどのような性質を持つように設計すべき?
- 特徴ベクトルの次元は大きい方が良い?(考慮する特徴量が多い方が良い?)
- 上記のようなラベル付け問題で正解画像群(各画像が100枚ずつなど)が多いと何が起きそう?

少し向きを変えて撮影した2枚の画像があります
左の画像の矩形領域と似た部分を右の画像から探してください

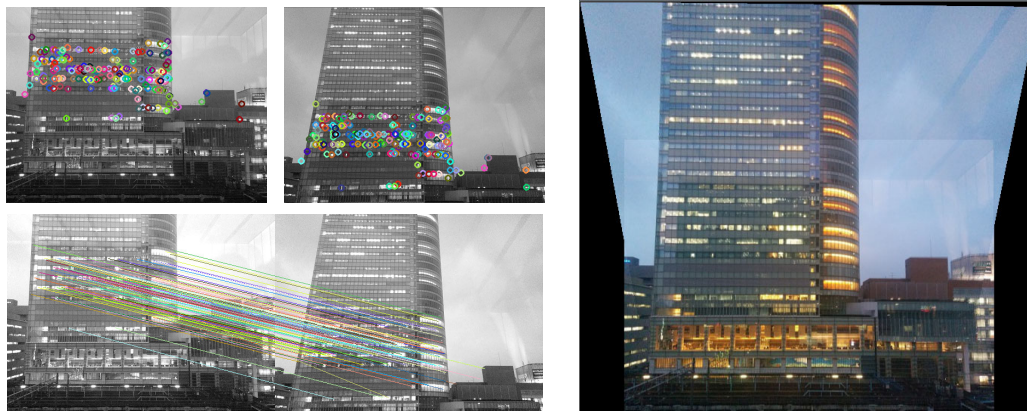


2枚の局所画像と似た部分を右から探して、という問題です



なぜそうわかったのでしょうか?
同じような検索をするプログラムを
作成できますか?

画像の局所領域の対応付けができるとパノラマ合成などができる
特徴的な点の取得 → 対応付け → 対応がピッタリ合うように変形



少し向きを変えて撮影した2枚の画像があります
左の画像の矩形領域と似た部分を右の画像から探してください

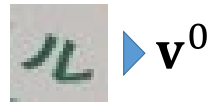


ル



どうやって検索しましたか?
どうしてこれが同じって言えるの?
大きさも角度も違うのに??
同じような検索をするプログラムを
作成できますか?

CVで用いられる検索方法（のイメージ）

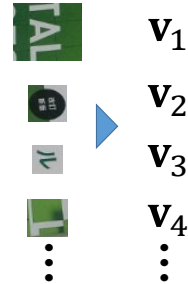


元の局所画像を特徴ベクトルに変換

例えばSIFT特徴なら128次元ベクトルに変換



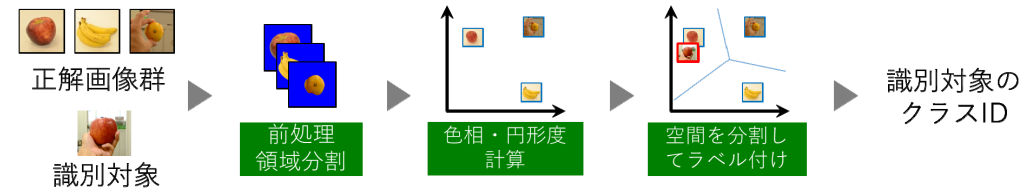
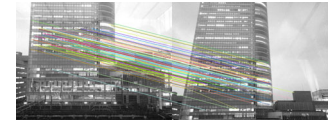
検索先の画像の局所領域についても同じ特徴ベクトルを計算
検索もとの特徴ベクトルと比較し似ているものを探す
SIFT特徴では、計算すべき窓の位置・サイズを厳選する手法も提案される



画像の局所領域に対して定義する特徴ベクトルを**局所特徴**と呼ぶこともあります
問) このような検索をするための特徴ベクトルはどのような性質を持つべきか述べよ

まとめ：解きたい問題（特徴ベクトルとは）

- 本講義では「計算機を利用して画像を理解する手法（Computer Vision）」を取り扱う
- 計算機により画像の全体や一部分を比較するために特徴ベクトルが利用される
- 特徴ベクトルの設計方法・利用方法が重要に



練習問題) 特徴ベクトルに関する設問に簡潔に回答せよ

画像分類をする際の特徴ベクトルはどのような性質を持つように設計すべき？

画像分類をする際の特徴ベクトルの次元は大きい方が良いか？（考慮する特徴量が多い方が良い？）

画像分類(ラベル付け問題)で正解画像群(各画像が100枚ずつなど)が多いと何が起きそう？

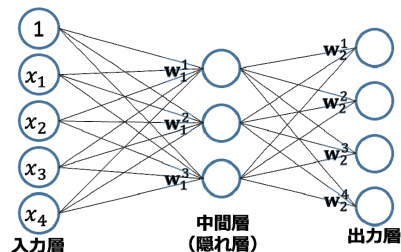
画像の局所領域の対応付けをする際の特徴ベクトルはどのような性質を持つべきか？

Contents

- 本講義で取り扱う問題の概要（特徴ベクトルとは）
- ニューラルネットワーク
- 各回で取り扱う内容の紹介

ニューラルネットワーク(NN)の概要

本講義では、古典的な手法を多く説明します（応用力を育む&NNの理解のために必要）
一方、実際には多くの問題についてNeural Networkを適用するのが主流です
講義中、なるべく最近の手法も（概要だけでも）説明を試みます



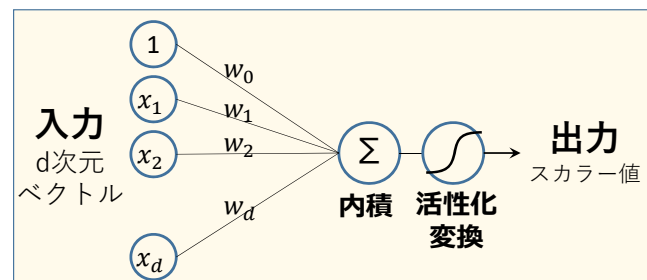
ニューラルネットワーク：

- ユニットや畳込み層からなる計算モデル
- 大量の教師データ（入力 $\mathbf{x}$ と出力 $\mathbf{y}$ のペア）をなるべく満たす関数 f を学習する

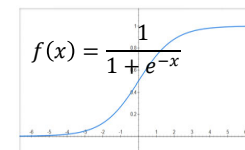
$$\mathbf{y} = f(\mathbf{x})$$

$\mathbf{x}, \mathbf{y}$ はベクトル（画像のことも）

ユニット(ニューロン)：NNを構成する最小単位



活性化関数には色々な非線形関数が利用される



シグモイド関数

- 重み係数 $\mathbf{w} = (w_0, w_1, \dots, w_d)$ と活性化関数 f を持つ
- ベクトル $\mathbf{x} = (1, x_1, \dots, x_d)$ を受け取り実数値を返す

計算過程

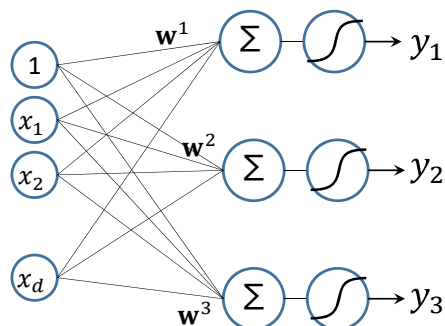
Step1 入力 $\mathbf{x}$ と重み係数の内積 $\mathbf{w}^T \mathbf{x}$ を計算

Step2 内積を活性化関数にかけて出力 $f(\mathbf{w}^T \mathbf{x})$

確認：重み係数 $\mathbf{w}$ を調整可能
教師データを満たすように重みを調整することを「学習」と呼ぶ

ユニットを並列につなぐ

複数ユニットを並列に並べると ベクトルを出力できるようになる



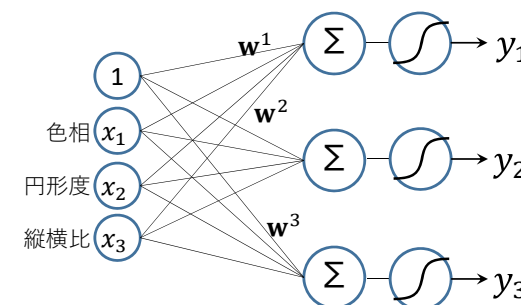
- 左図は、 d 次元ベクトル $\mathbf{x}$ を受け取り、3次元ベクトル $\mathbf{y}$ を出力するNN
- 並列に繋いだユニットをまとめて層（全結合層）とよぶ

問) 左図のNNにおいて調製すべき重み係数は何個ある？

答え： $(d+1) * 3$ 個

ユニットを並列につなぐ：応用例のイメージ

例) 画像をリンゴ・バナナ・みかんに分類せよ



- 教師画像（ラベル付き画像）は大量に与えられる
- 各画像から（色相、円形度、縦横比）という3次元特徴ベクトルを取り出せるとして

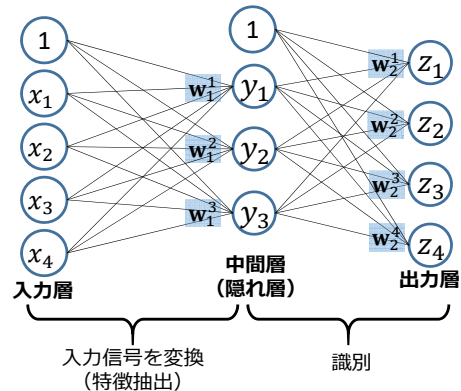
リンゴの特徴ベクトル $\mathbf{x}$ を入れたら(1,0,0)
バナナの特徴ベクトル $\mathbf{x}$ を入れたら(0,1,0)
ミカンの特徴ベクトル $\mathbf{x}$ を入れたら(0,0,1)
を出力するように訓練する

※ この例では特徴ベクトルを抽出し、それをNNに入力しています。
が、実際には人がデザインした特徴をNNに入れるのは主流ではありません

ユニットを直列につなぐ

入力層と出力層の間に中間層をはさみこむと入力を識別しやすい形に変換してから識別する事が可能に

→ 線形分離不可能な問題にも対応できる（後々、丁寧に説明します）

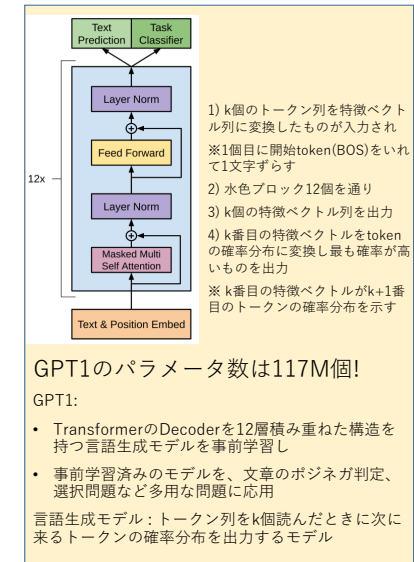
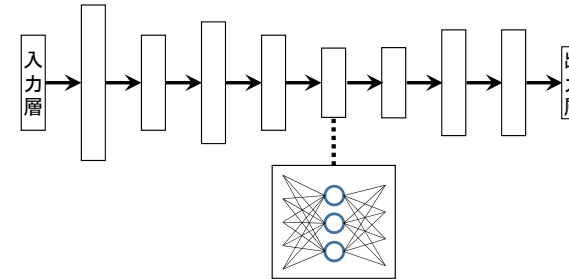


左図では非線形関数を省略
重み係数が未知数で、これを教師データから学習する
→左図では $5 \times 3 + 4 \times 4 = 31$ 個の未知数

30

深層学習 : Deep learning

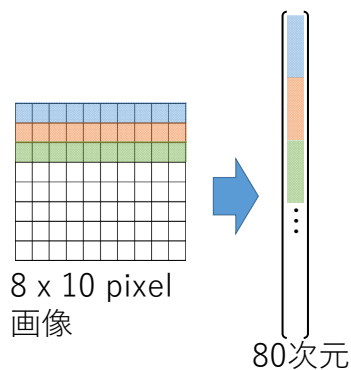
- 7層 ~ 20層など、多層構造を持つNNを用いて学習する手法（と少し前は言われていましたが、最近はもっと多層）
- 特徴抽出を繰り返し、最後に識別を行なうと考えていいかも



31

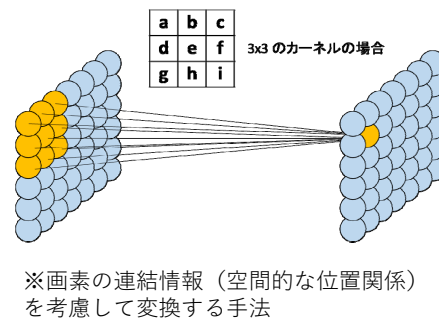
画像データをNNに入力する

方法1) 画素値をベクトルに変換



※この方法は使わないことはないが主流ではない

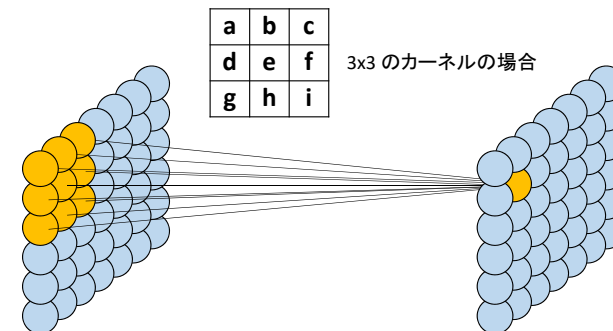
方法2) 畳み込みニューラルネットワーク (CNN) を利用



※画素の連結情報（空間的な位置関係）を考慮して変換する手法

Convolutional Neural Network

- 主に画像データに対して用いられるNeural Networkの一種
- 畳み込み（線形フィルタ）で入力画像を出力画像に変換
- フィルタの重みを変数で、これを訓練データより調整（学習）する



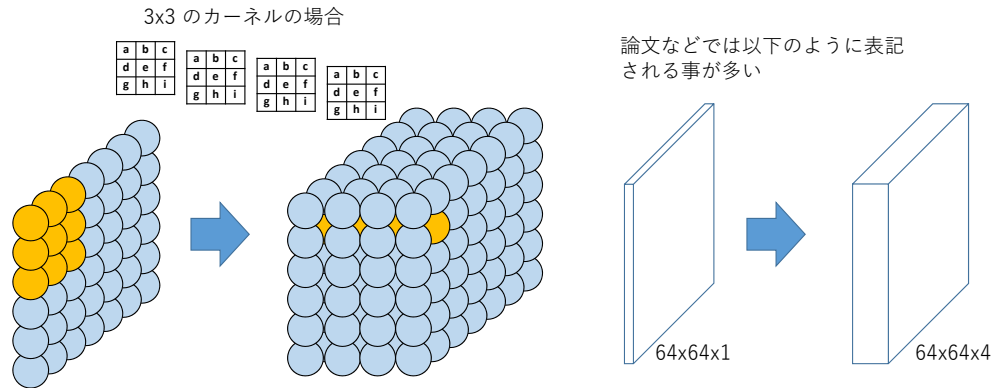
左図では省略していますが畳み込み計算の後、各画素に活性化関数（非線形関数）を適用します

33

Convolutional Neural Network

線形フィルタを4個用意すると4枚の画像が出力される

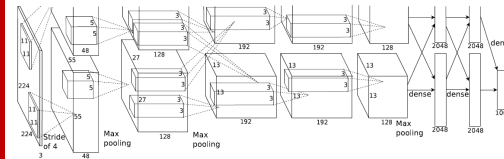
→ channel方向に積み重ねて表現される



Convolutional Neural Network の例

AlexNet (NIPS 2012)

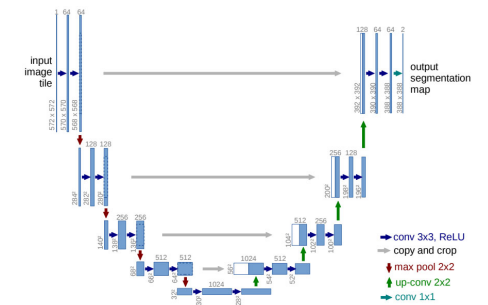
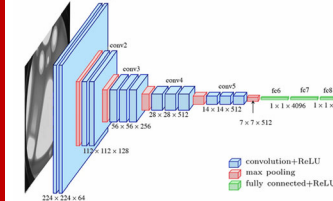
CNN+全結合層で画像のクラス分類



VGG (ICLR 2015)

CNN+全結合層で画像のクラス分類

フィルタサイズを小さく(3x3)にする代わりに多層化することで性能向上



UNet (MICCAI ワークショップ 2015)

CNNとプーリング層・Up-convolutionのみで構成される領域分割用のモデル

同じレイヤにSkip接続を配置することで精度向上を実現

NIPS2012

ImageNet Classification with Deep Convolutional Neural Networks

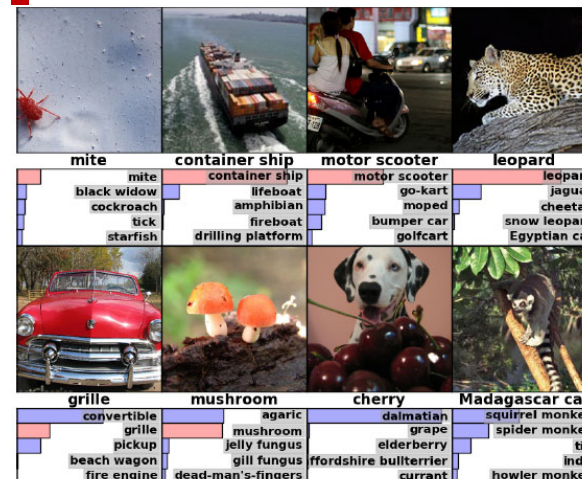
Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilva@cs.utoronto.ca

Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

- CV分野におけるDeep neural network流行の火付け役の筆頭 『AlexNet』
- ILSVRC（画像認識コンテスト）にて、従来法（SIFT+SVMなど）に圧勝
- 画像を1000個のクラスに分類するタスクを解く

[論文 Figure 4より引用]



解きたい問題：画像識別問題

画像内の物体を1000個の選択肢から回答する

※ 1000クラスは結構偏りが大きい。犬種だけで100クラス以上など。世界中のあらゆるものを分類というものではないので注意。

左図の各ブロックは、、、

+ テスト画像とその正解ラベル

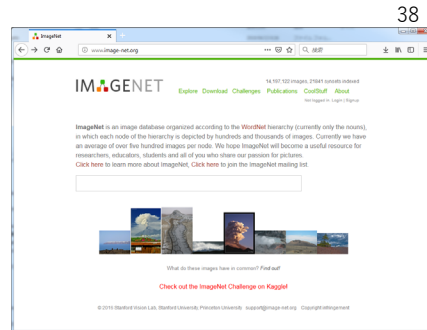
+ AlexNetの出力結果の上位 5 位

かなりややこしい画像についてもtop5位の中に正解がある

Dataset

• ImageNetを利用.

- 1.5×10^6 枚のラベル付ハイレゾ画像
- 22,000カテゴリに分類される
- ILSVRCではこの一部を利用
 - 1000カテゴリ, 各カテゴリ1000枚程度
 - 合計 1,200,000枚の画像を訓練データとする

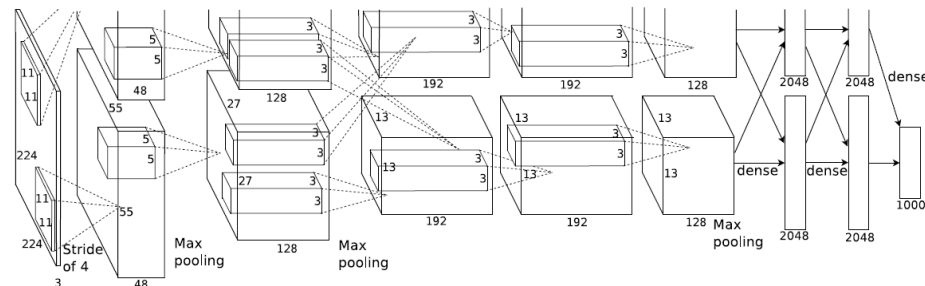


• 多様な解像度の画像が含まれる → 256x256に変換する

- 横方向の解像度が256になるようにscalingし、256x256を切り出す
- RGBカラー画像を利用（各画素から平均値を引き算しておく）

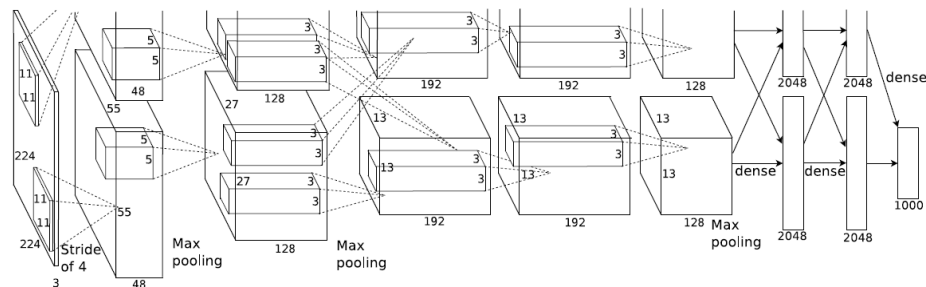
The Architecture

[論文 Figure 2より引用]



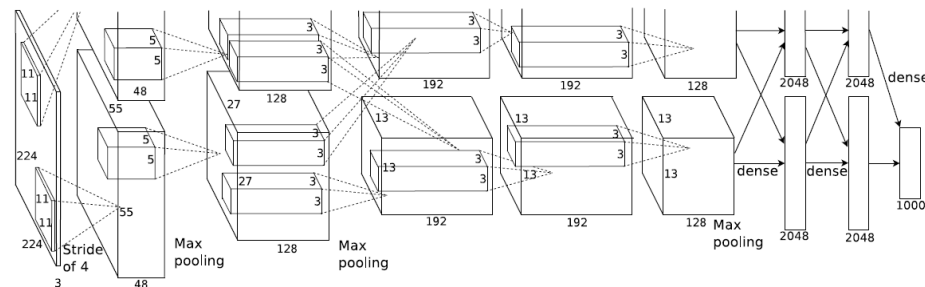
- 畳み込み5層, 全結合3層, 最後に1000次元のsoft maxを出力
- 上と下の2パスが存在するが, これはそれぞれが異なるGPUにはいるように設計されている
- 2, 4, 5層目は同じGPU内の層とつながっている
- 1, 3層目は, 両方のGPU内の層とつながっている

The Architecture



- 第1層: 入力画像 224x224x3に対して, 11x11x3のカーネル96個を適用する (半分が上, 半分が下). ストライドは4 pix.
- 第2層: 5x5x48のカーネル256個をかける.
- 第3層: 3x3x256のカーネル384個をかける. 入力が128+128チャンネル扱い
 - Normalize + poolingも行なう
- 第4層: 3x3x192のカーネル384個をかける.

The Architecture



- 第5層: 入力map 13x13x384に対して, 3x3x192のカーネル256個を適用する
- 第6,7層は 4096個のニューロンからなる全結合層
- 第8層は1000個のニューロンからなる全結合層 (出力はsoft max)

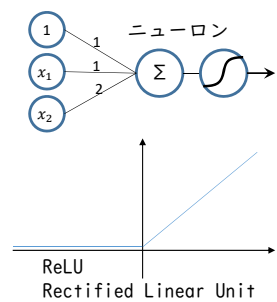
練習問題

(1) 2次元ベクトルを入力とするニューロン（ユニット）があり、重みは $(1, 1, 2)$ である。また、活性化関数にはReLUが設定されている。このニューロンに $(1, -2)$ 、 $(3, -1)$ を入力した際の出力を求めよ

ReLU関数：
$$f(x) = \max(x, 0) = \begin{cases} 0 & \text{if } x < 0 \\ x & \text{otherwise} \end{cases}$$

(2) 特徴ベクトルを人間が設計する時代の画像認識手法と比べて、AlexNetのようなNNを利用した手法が画期的であった理由を以下よりすべて選択せよ

- A. 認識精度の向上には、SVM（サポートベクターマシン）などを用いて特徴空間の分割方法を学習することが重要であると証明した
- B. 特徴抽出を人間が設計するのではなく教師データから自動的に学習する様になった
- C. ニューラルネットワークはあらかじめ注意深く設計された特徴ベクトルを入力することで大幅な性能向上が起こることを示した
- D. 入力から出力までを一つのネットワークで完結させる「End-to-End」の学習が可能になった
- E. 画像の局所的な特徴を抽出するために、全結合層のみを用いたネットワーク構造が最適であることを示した



Contents

- 本講義で取り扱う問題の概要（特徴ベクトルとは）
- ニューラルネットワーク
- 各回で取り扱う内容の紹介

予習：講義の目次

- 01. 序論：ガイダンス、深層学習による画像処理
- 02. 特徴検出1：テンプレートマッチング、コーナー検出、エッジ検出
- 03. 特徴検出2：ハフ変換、DoG, SIFT特徴
- 04. 領域分割：領域分割とは、閾値法、領域拡張法、グラフカット法、
- 05. オプティカルフロー：領域分割残り、Lucas-Kanade法
- 06. パターン認識基礎1：パターン認識概論、サポートベクタマシン
- 07. パターン認識基礎2：ニューラルネットワーク、深層学習
- 08. パターン認識基礎3：主成分分析、オートエンコーダ
- 09. 筆記試験
- 10. プログラミング演習 1: PC室
- 11. プログラミング演習 2: PC室
- 12. プログラミング演習 3: PC室
- 13. プログラミング演習 4: PC室
- 14. プログラミング演習 5: PC室

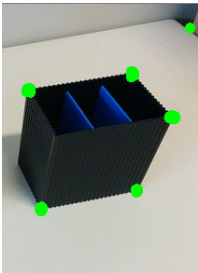
予習 2週目：テンプレートマッチング、エッジ検出、コーナー検出



入力画像



テンプレート



Harrisのコーナー検出

- 画像から角を毛出する手法
- 角らしさを表す特徴量を抽出



Cannyのエッジ検出

- 画像からエッジを抽出する
- 微分フィルタを活用して細かいエッジを出力する

テンプレートマッチング

- 物体検出のための最も初歩的な手法
- 検出対象の画像（テンプレート）を用意し、入力画像をラスタスキャンし相違度を評価
- 相違(類似)度：SAD, SSD, NCCなど

YOLOによる物体検出

今回は最も単純な物体検出手法であるテンプレートマッチングを紹介（色々応用可能だが…）

現在は物体検出をやるならNNを使うのが普通 → YOLO を動かしてみる

Step0 仮想環境を作ってactivate（飛ばしてもOK）

フォルダを作成しそこをcurrentにするcommand promptで

```
> python -m venv cv_sample  
> .\cv_sample\Scripts\activate.bat
```

Step1 必要なライブラリをインストール

```
> python -m pip install opencv-python  
> python -m pip install ultralytics
```

Step2 右のコードを実行

```
> python YOLOtest.py
```



```
import cv2  
from ultralytics import YOLO  
  
# モデルロード（軽量なものを利用、ここで2.6M程度のDLが入る）  
model = YOLO("yolo11n.pt")  
  
cap = cv2.VideoCapture(0)  
  
if not cap.isOpened():  
    print("No camera")  
    exit()  
  
while True:  
    ret, frame = cap.read()  
    if not ret: break  
  
    # 推論して結果を表示  
    results = model(frame, stream=False)  
    annotated_frame = frame  
    if len(results) > 0:  
        annotated_frame = results[0].plot()  
    cv2.imshow("YOLO11", annotated_frame)  
  
    if cv2.waitKey(1) & 0xFF == ord("q"): break  
  
cap.release()  
cv2.destroyAllWindows()
```

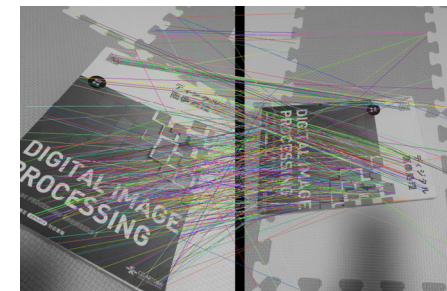
予習 3週目：局所特徴(SIFT)、Hough変換

• 特徴ベクトルとは何か

- 検出された特徴点同士を比較するため、特徴点周囲の局所領域をベクトルの形で表すもの。

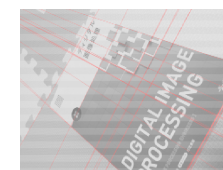
• SIFT特徴

- DoGの極値を特徴点として検出
- 特徴点のスケールに応じた局所領域を考慮
- 特徴点周囲の勾配方向に沿って局所窓を回転
- 局所窓を4分割し、各領域の勾配ヒストグラムを特徴ベクトルとする



• Hough変換

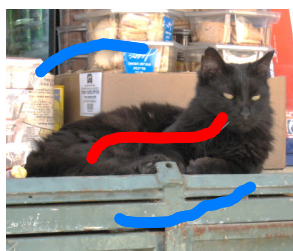
- 直線検出のための有名なアルゴリズム
- プログラミング課題でも出題予定



予習 4週目：領域分割（下図はグラフカット法）

GraphCut.exe

全景制約 背景制約

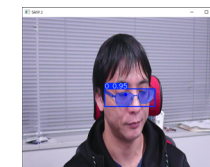


前景・背景に属する画素を適当に入力すると、これを制約に画像を二値化
二値化をエネルギー最小化問題として定式化し、フローネットワークの最小
カットにより最適解を計算する

臓器などの塊状領域に対してはかなり強力な領域分割法

SAM（Segment Anything Model）

SAM（Segment Anything Model）を動かしてみる
画面中央の点をシード（プロンプト）としてここからの連結領域の分割が可能



Step0 仮想環境をactivate（飛ばしてもOK）

フォルダを作成しそこをcurrentにするcommand promptで

```
> .\cv_sample\Scripts\activate.bat
```

Step1 必要なライブラリをインストール

```
> python -m pip install opencv-python  
> python -m pip install ultralytics
```

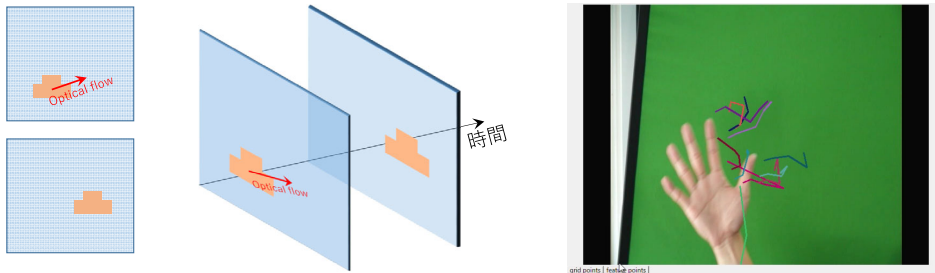
Step2 右のコードを実行

```
> python samtest.py
```

```
import cv2  
from ultralytics import SAM  
  
# SAM 2 をロード  
model = SAM("sam2_b.pt")  
cap = cv2.VideoCapture(0)  
  
while cap.isOpened():  
    success, frame = cap.read()  
    if not success:  
        break  
  
    # ここがプロンプト  
    h, w, _ = frame.shape  
    center_point = [w // 2, h // 2]  
  
    results = model.predict(source=frame, points=[center_point], labels=[1], device="cuda")  
  
    annotated_frame = results[0].plot()  
    cv2.circle(annotated_frame, tuple(center_point), 5, (255, 0, 0), -1)  
    cv2.imshow("SAM 2", annotated_frame)  
  
    if cv2.waitKey(1) & 0xFF == ord("q"):  
        break  
  
cap.release()  
cv2.destroyAllWindows()
```


予習 5週目：オプティカルフロー

- 2枚の画像内におけるある物体の移動量をベクトルとして表現したもの
- 動画内の『物体追跡』や『物体の動きの解析』などに利用される
- ブロックマッチング法, Lucas-Kanade法など



RAFTによるOptical flow計算

現在はOptical Flow計算にも便利なNNモデルが利用可能 → RAFTを動かしてみる

Step0 仮想環境をactivate (飛ばしてもOK)

フォルダを作成しそこをcurrentにするcommand promptで

```
> .\cv_sample\Scripts\activate.bat
```

Step1 必要なライブラリをインストール

Cuda + torchを使う場合は <https://pytorch.org/> からインストールコマンドを取得してインストール

```
> pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

CPUでよければ以下でもOK

```
> pip install torch torchvision numpy opencv-python
```

Step2 右のコードを実行

```
> python rafttest.py
```

```
import cv2
import torch
from torchvision.models.optical_flow import raft_small, Raft_Small_Weights
from torchvision.models.optical_flow import raft_large, Raft_Large_Weights
from torchvision.utils import flow_to_image

# Prepare the model
device = "cuda" if torch.cuda.is_available() else "cpu"
model = raft_small(weights=Raft_Small_Weights.DEFAULT).to(device).eval()
# 大きいモデルを動かすなら以下
# model = raft_large(weights=Raft_Large_Weights.DEFAULT).to(device).eval()

# prepare the video capture
cap = cv2.VideoCapture(0)
ret, frame1 = cap.read()

while ret:
    ret, frame2 = cap.read()

    # 画像をリサイズしてTensorに、さらに[-1,1]に正規化
    img1 = torch.from_numpy(cv2.resize(frame1, (640, 480))).permute(2,0,1).float().to(device)
    img2 = torch.from_numpy(cv2.resize(frame2, (640, 480))).permute(2,0,1).float().to(device)
    img1 = (img1 / 127.5) - 1.0
    img2 = (img2 / 127.5) - 1.0
    with torch.no_grad():
        flow = model(img1, img2)[1:-1]

    # 可視化画像の生成 (flow image関数をそのまま利用)
    flow_img = flow_to_image(flow)[0].permute(1,2,0).cpu().numpy()
    flow_img = cv2.cvtColor(flow_img, cv2.COLOR_RGB2BGR)
    cv2.imshow("RAFT Flow", flow_img)

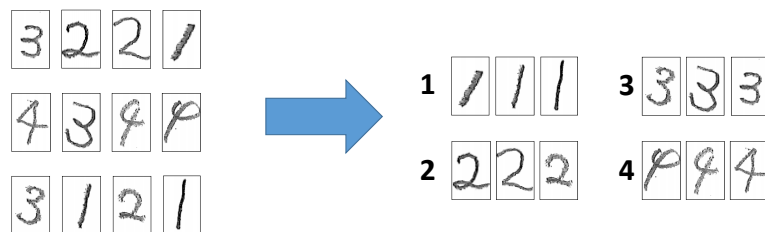
    frame1 = frame2
    if cv2.waitKey(1) & 0xFF == ord('q'): break

cap.release()
cv2.destroyAllWindows()
```

予習 6週目：パターン認識

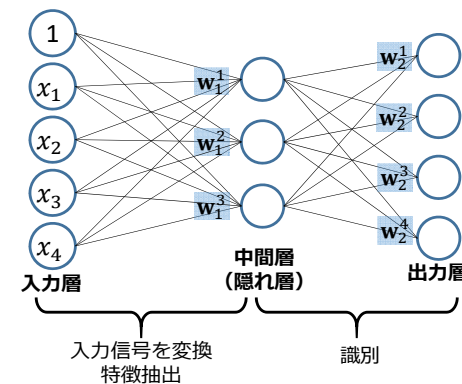
『データの中の規則性を自動的に見つけ出し、その規則性を使ってデータを異なるカテゴリに分類する処理』 (PRML, C.M. Bishop)

例) 手書き文字画像の認識



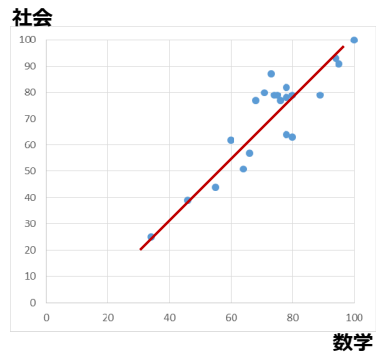
予習 7週目：ニューラルネットワーク

- 複数のユニット（ニューロン）を層状につないだネットワーク
- 入力層が特徴ベクトルを受け取り、出力層から識別結果が出力される

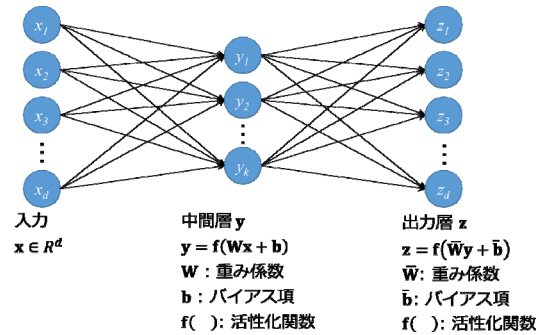


※ ここではニューラルネットワークの計算法を扱いません
 ※ 学習方法（誤差逆伝搬）の詳細、様々な学習方法（Drop out、バッチ正規化）、活性化関数、ネットワーク構造（RNN、LSTM、Attention）については扱いません。

予習 8週目：主成分分析とオートエンコーダ



38



まとめ

- 本科目の実施方法
 - 01~08回目：対面 + 小テスト
 - 09 回目：筆記試験
 - 10~14回目：プログラミング演習
- 本講義で扱う課題（特徴ベクトルとは）
- ニューラルネットワークの概要
- 講義の簡単な予習

前期14週の間、よろしくおねがいします！